

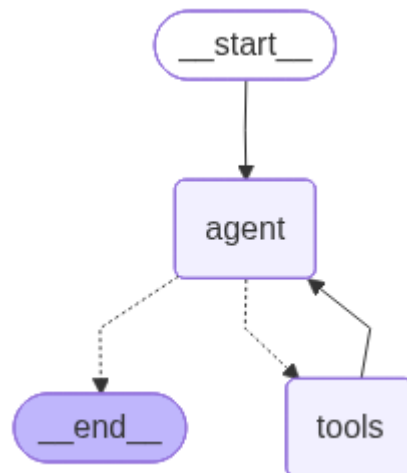
HW Milestones — Agentic RAG

Seven milestones. Test as you go using the `TEST_TO_RUN` switch in `main.py`.

`prompts.py` and `config.py` are provided. Don't change them.

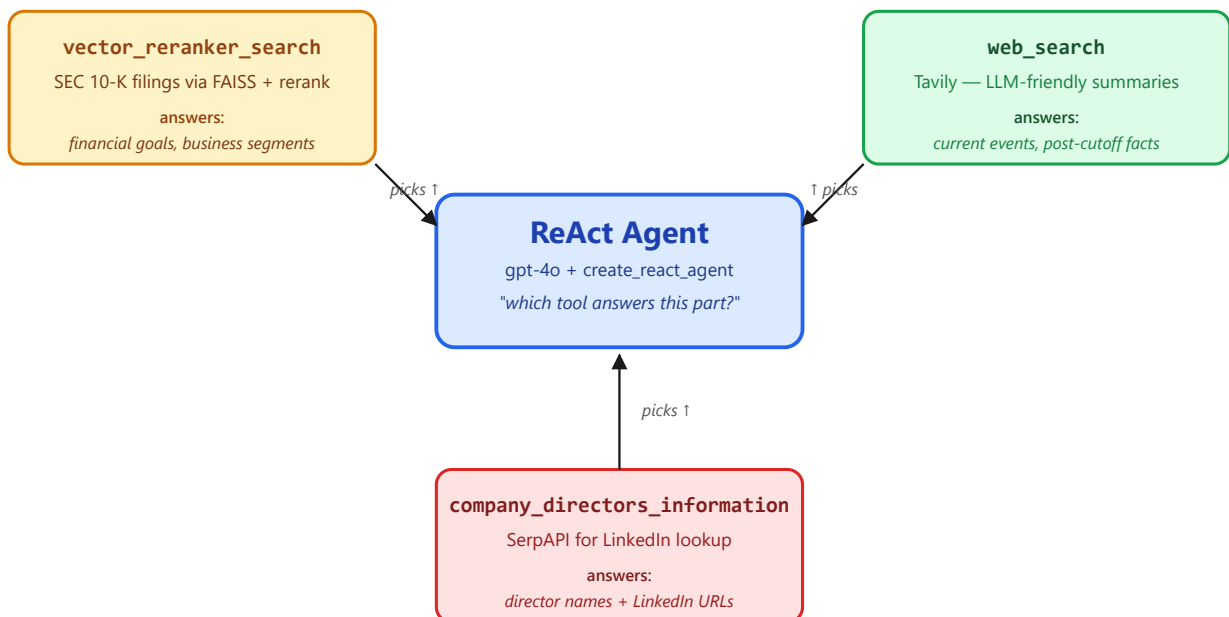
For setup, submission, rubric, and pitfalls, see [../HW_INSTRUCTIONS.md](#).

What you're building — by the end of M7, your graph will look like this:



The agent picks among three tools to answer different parts of the demo question:

Central agent picks among three tools based on the question



M1 — Read what's provided (no code, ~10 min)

Open `prompts.py`. Two prompts and one demo question.

- `REACT_PROMPT` — the standard ReAct prompt template (Thought / Action / Observation / Final Answer).
- `NAME_EXTRACTION_PROMPT` — instructs `gpt-4o-mini` to pull director names out of an SEC 10-K excerpt as a comma-separated list.
- `DEMO_QUESTION` — the multi-part Tesla question. The agent has to figure out which tools to use to answer each part.

Open `config.py`. The `CONFIG` dict has all hyperparameters: chunk size, model names, URLs, top-K values, tool descriptions. Other files import `CONFIG` and read what they need.

You're not changing either file. Read them so you understand what's in scope.

M2 — Read what's pre-filled in the TODO files (no code, ~25 min)

Each of the four TODO files has substantial pre-filled scaffolding. The pieces below are tricky and would derail you if you had to invent them. Pre-fixed for you — read and understand, don't try to redo.

A — The flashrank monkey-patch (`retriever.py`)

```
from flashrank import Ranker, RerankRequest
import langchain_community.document_compressors.flashrank_rerank as fr_mod
fr_mod.RerankRequest = RerankRequest
from langchain_community.document_compressors import FlashrankRerank
```

Real-world version-skew bug: `langchain_community.document_compressors.flashrank_rerank` looks for `RerankRequest` in the wrong place. This snippet rebinds it. Without it, `FlashrankRerank` import fails. Will get fixed upstream eventually; for now this is what production code looks like.

B — The `BaseTool` class skeletons (`tools.py`)

Each of your three tools subclasses LangChain's `BaseTool`:

```
class MyTool(BaseTool):
    name: str = "snake_case_name"    # NO SPACES – OpenAI's tool-calling API rejects them
    description: str = "tells the agent when to use this tool"

    # ... custom attributes ...

    def __init__(self, ...):
        super().__init__(...)

    def _run(self, query: str) -> str:
        ...    # this is what you implement
```

The classes, names, descriptions, and `__init__`s are all written. You only fill in `_run` (and a couple of helper methods on `CompanyDirectorsTool`). The **description is critical** — the agent reads it to decide when to call your tool.

C — `from langgraph.prebuilt import create_react_agent` (`main.py`)

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(llm, tools)
result = agent.invoke({"messages": [HumanMessage(content=DEMO_QUESTION)]})
```

This is **the LangGraph version** of the agent loops you built by hand in Reflection and Reflexion. Same loop pattern (LLM → tool → LLM → tool → ... → final answer), but pre-baked. The signature is `{"messages": [HumanMessage(...)]}` — same as Reflection. The result has the full conversation including tool-call `AIMessages` and `ToolMessages` with observations.

D — The `User-Agent` header (`ingestion.py`)

```
loader = webBaseLoader(url, header_template={"User-Agent": user_agent})
```

SEC.gov blocks requests with the default urllib User-Agent. We pass a custom one via `header_template`. Forget this and the loader silently returns empty docs.

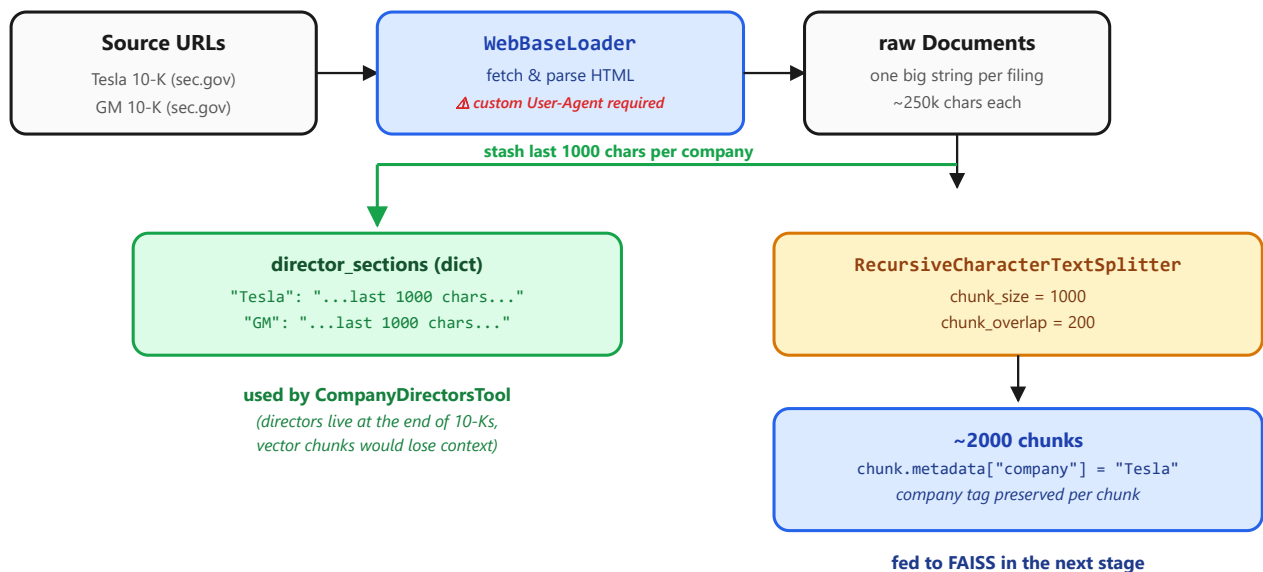
Now you're ready to fill in TODOs. The starter ships with `TEST_TO_RUN = "ingestion"` already set — just run `uv run python main.py` to start hitting milestones.

Each milestone below describes what your code should do. The starter `.py` files have detailed step-by-step docstrings inside each TODO function — those are your second source of guidance. Translating English description into Python is part of the work.

M3 — `ingestion.py`: load and split SEC filings

Goal: turn two URLs into a list of chunks (each tagged with company metadata) and a dict of "last 1000 chars per company" for the directors tool to use later.

Build the knowledge base once: load filings, chunk, tag — plus stash director sections separately



Fill in `load_and_process_filings`:

Build a `RecursiveCharacterTextSplitter` from the chunk size and overlap params. Loop over the `(company, url)` tuples in `urls`. For each:

- Build a `webBaseLoader` (passing `user_agent` via `header_template` — the param signature is provided)
- Call `loader.load()` to get a list of `Document` objects
- Stash the last 1000 chars of `docs[0].page_content` into `director_sections[company]` — director listings live at the end of 10-K filings
- Use `splitter.transform_documents(docs)` to chunk
- Tag each chunk with `chunk.metadata["company"] = company` so the agent can tell Tesla chunks from GM chunks
- Append all chunks to a running list

Return `(all_chunks, director_sections)`.

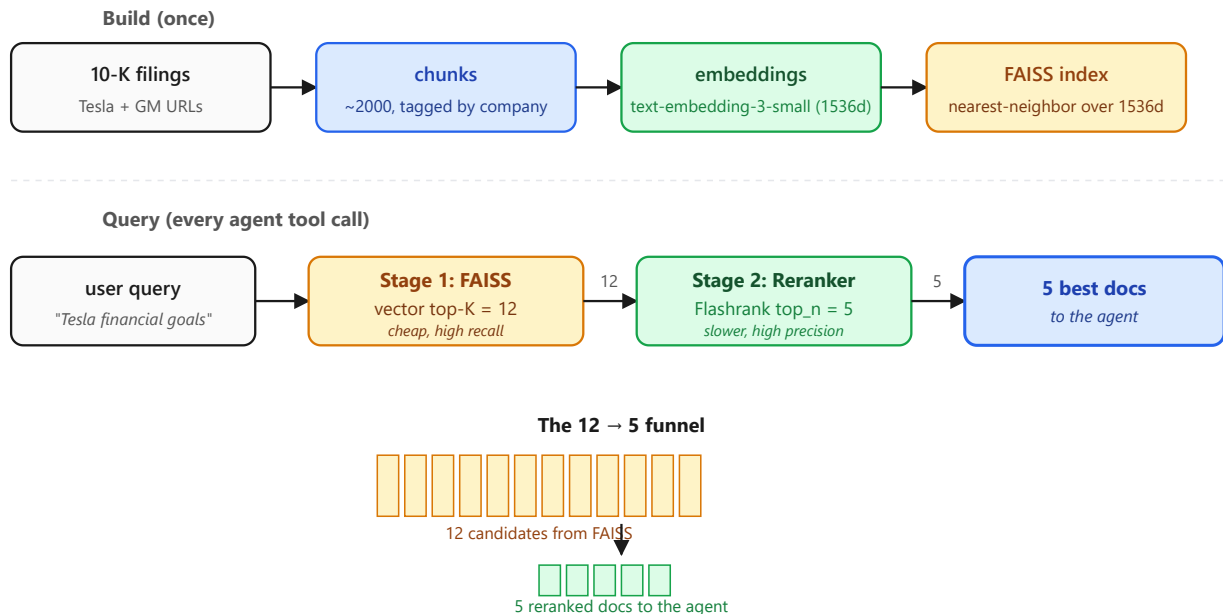
Add `print` statements as you go — visible progress helps when ingestion is slow.

Test: `TEST_TO_RUN = "ingestion"`, run `uv run python main.py`. Expected: ~2000 chunks total, two companies in `director_sections`.

M4 — `retriever.py`: vector store + reranker

Goal: take chunks, build a FAISS index, wrap with Flashrank.

Two-stage retrieval: cheap recall first, then quality rerank



```
create_vector_store(chunks, embedding_model)
```

Two lines of real work: instantiate `OpenAIEmbeddings` with the model name, then `FAISS.from_documents(chunks, embeddings)`. Return the FAISS object. Add a print line showing `vectorstore.index.ntotal` and `.index.d` so you can verify the build succeeded.

```
create_retriever_with_reranking(vectorstore,
num_retrieved, reranker_model, num_reranked)
```

A four-step pipeline:

1. Get a base retriever from the vectorstore: `as_retriever(search_kwargs={"k": num_retrieved})`. This gives you the top `num_retrieved` chunks for any query (default 12).
2. Build a `Ranker` client (Flashrank): `Ranker(model_name=reranker_model)`.
3. Wrap the client in a `FlashrankRerank` compressor with `client=`, `model=`, and `top_n=num_reranked` (default 5). The reranker shrinks the 12 candidates down to the 5 best ones for the query.
4. Combine the base retriever and the reranker into a `ContextualCompressionRetriever` (`base_retriever=...`, `base_compressor=...`). Return that.

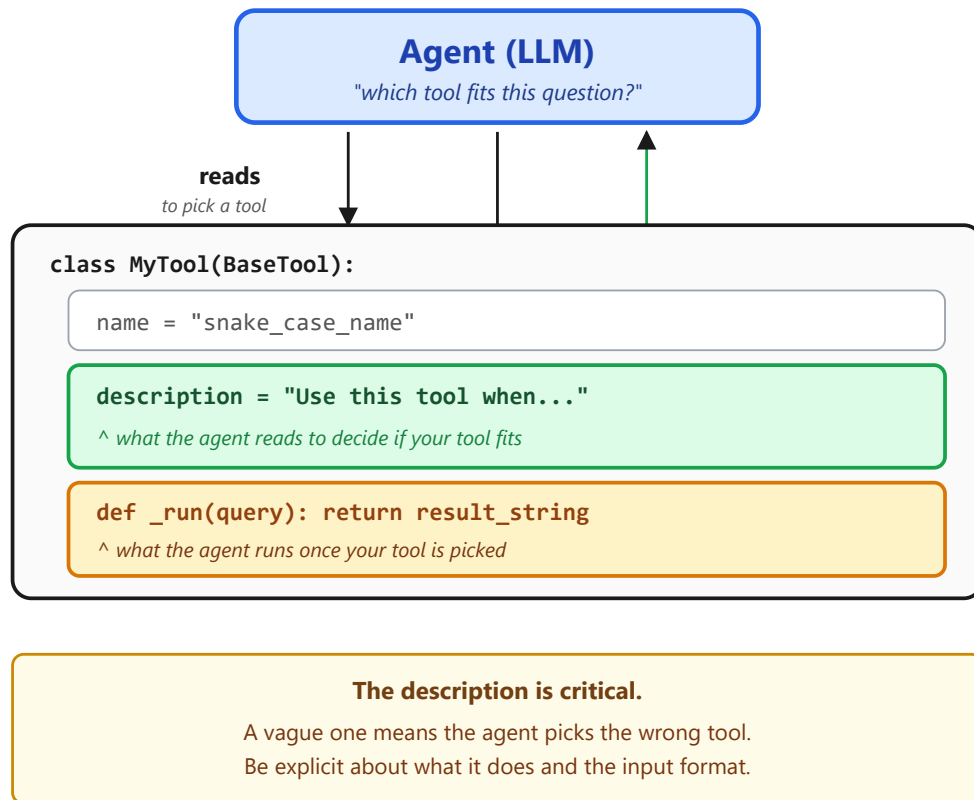
This is the "two-stage retrieval" pattern: cheap candidate fetch, then high-quality rerank.

Test: `TEST_TO_RUN = "retriever"`. Expected: a FAISS index with ~2027 vectors at dim 1536, then 5 reranked docs returned for the sample query.

M5 — `tools.py`: three `BaseTool` subclasses

Goal: build three tools the agent can pick between.

A `BaseTool` subclass has three parts. The agent uses two of them.



The class skeletons are provided (you saw them in M2.B). You write only the `_run` method body for each tool plus the helper methods on `CompanyDirectorsTool`. Each starter docstring lists the steps; this milestone gives the conceptual story.

M5.1 — `VectorRerankerSearchTool._run`

Call `self.retriever.invoke(query)` to get reranked docs. Format the top `self.num_results` of them: each one becomes the company tag from `doc.metadata["company"]` followed by a newline followed by `doc.page_content`. Join all formatted entries with `"\n\n"` (double newline separator). Wrap the whole thing in try/except and return an error string on failure — the agent's error message is your only feedback signal.

M5.2 — `webSearchTool._run` (Tavily)

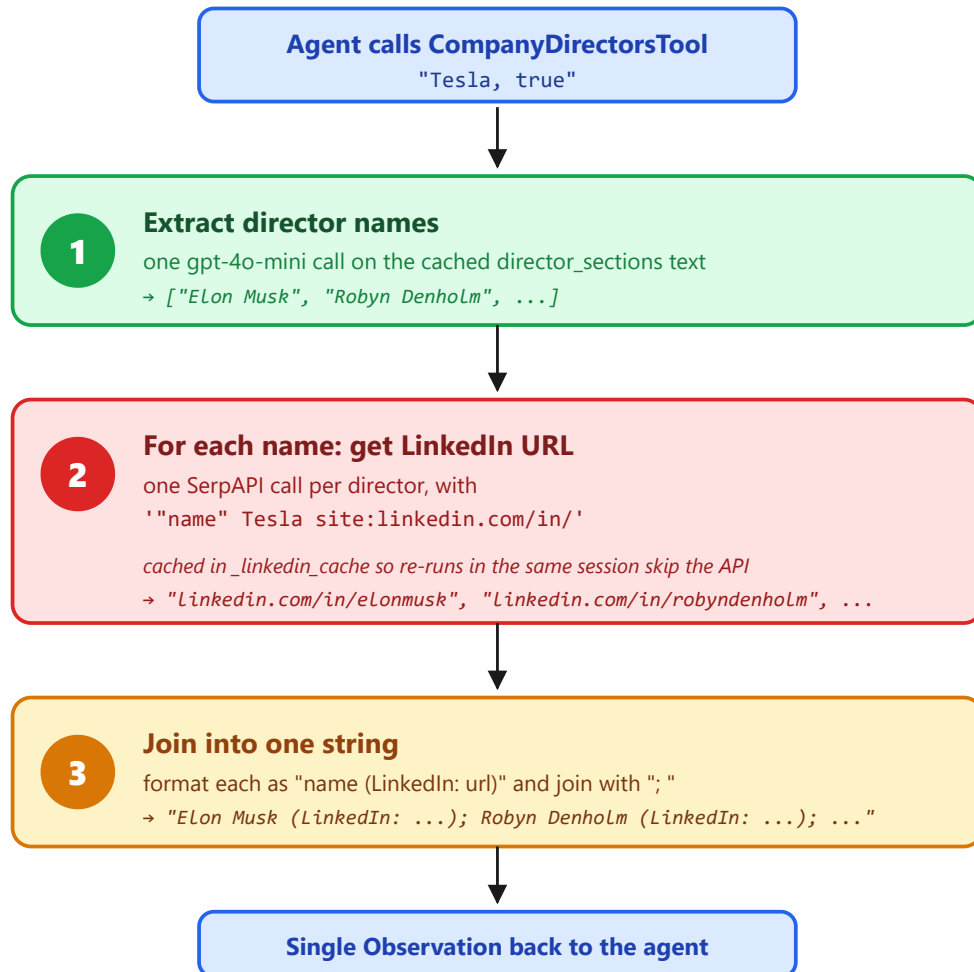
Invoke the Tavily `self.search` with `{"query": query}` (same shape as Reflexion). The result is a dict; the relevant data is in `results["results"]` — a list of dicts each with `title`, `content`, and `url` fields. Format the top `self.num_results` of them as something like:

```
Title: <title>  
Snippet: <content>  
Link: <url>
```

Join with double newlines. Return "No web results found." if the list is empty. Try/except wrap.

M5.3 — CompanyDirectorsTool — three methods

One tool call expands into an LLM extraction + N web lookups



`_run`

Parse the input string `"Tesla, true"` into `company` and `include_linkedin` (split on comma, lowercase the second part, treat missing as `True`).

Look up `self.director_sections.get(company)`. Return an error string if it's `None`.

Call `self._extract_director_names(section)` to get the list of names. If empty, return another error string.

If `include_linkedin` is `True`: for each name, call `self._get_linkedin_handle(name, company)` and format as `"{name} (LinkedIn: {handle})"`. Join with `"; "` and return.

Otherwise: just return the names joined with `", "`.

Wrap the whole thing in try/except.

`_extract_director_names`

This is the same `prompt | llm | parser` pattern as Reflexion, but with `CommaSeparatedListOutputParser` instead of `with_structured_output`. Build a `ChatOpenAI` (use `self.extraction_model` and `self.extraction_temperature`), a `CommaSeparatedListOutputParser`, and a `PromptTemplate.from_template(NAME_EXTRACTION_PROMPT)`. Pipe them together. Invoke with `{"text": text}`. Return the resulting list of strings.

Try/except — return `[]` if parsing fails so the tool degrades gracefully.

`_get_linkedin_handle`

Three things to do, in order. Cache check first (saves SerpAPI quota), SerpAPI call, cache store. All wrapped in try/except.

Cache key: a string that uniquely identifies a (name, company) pair. Pop into the module-level `_linkedin_cache` (already imported for you).

If not cached: build a `SerpAPIWrapper`, call `.results()` with the query `f'"{name}" {company} site:linkedin.com/in/'` — the `site:` operator is what scopes results to actual LinkedIn profiles, not articles about people. The result dict has an `organic_results` list; pluck the `link` from the first entry. Default to `"Profile not found"` if the list is empty or the link is missing.

Cache before returning.

The cache and try/except aren't optional — without the cache you'll burn SerpAPI quota; without try/except a single failed API call kills the whole agent run.

Test: `TEST_TO_RUN = "tools"`. All three tools fire independently. Expected: retriever returns 10-K excerpts; web search returns Tavily results for an auto-show query; directors tool returns 8 Tesla directors with LinkedIn URLs.

⚠ This burns ~8 SerpAPI calls (one per Tesla director). Make sure M3, M4, M5.1, M5.2 work first before testing M5.3.

M6 — `main.py`: orchestration helpers

Goal: wire ingestion + retriever + tools + agent into helper functions.

`build_pipeline()`

Calls `load_and_process_filings`, then `create_vector_store`, then `create_retriever_with_reranking` — passing the right `CONFIG` keys to each. Returns the tuple `(retriever, director_sections)` for downstream use. Most of this is pasting the right `CONFIG["..."]` values to the right kwargs.

`build_tools(retriever, director_sections)`

Returns a list of three tool instances, one each of `VectorRerankerSearchTool`, `WebSearchTool`, and `CompanyDirectorsTool`. Each constructor takes the retriever or director sections plus the relevant `num_results` / `extraction_model` / `extraction_temperature` keys from `CONFIG`.

`build_agent(tools)`

Two lines: instantiate `ChatOpenAI` with `CONFIG["reactModelName"]` and `CONFIG["reactModelTemperature"]`, then return `create_react_agent(llm, tools)` (already imported at the top of `main.py`).

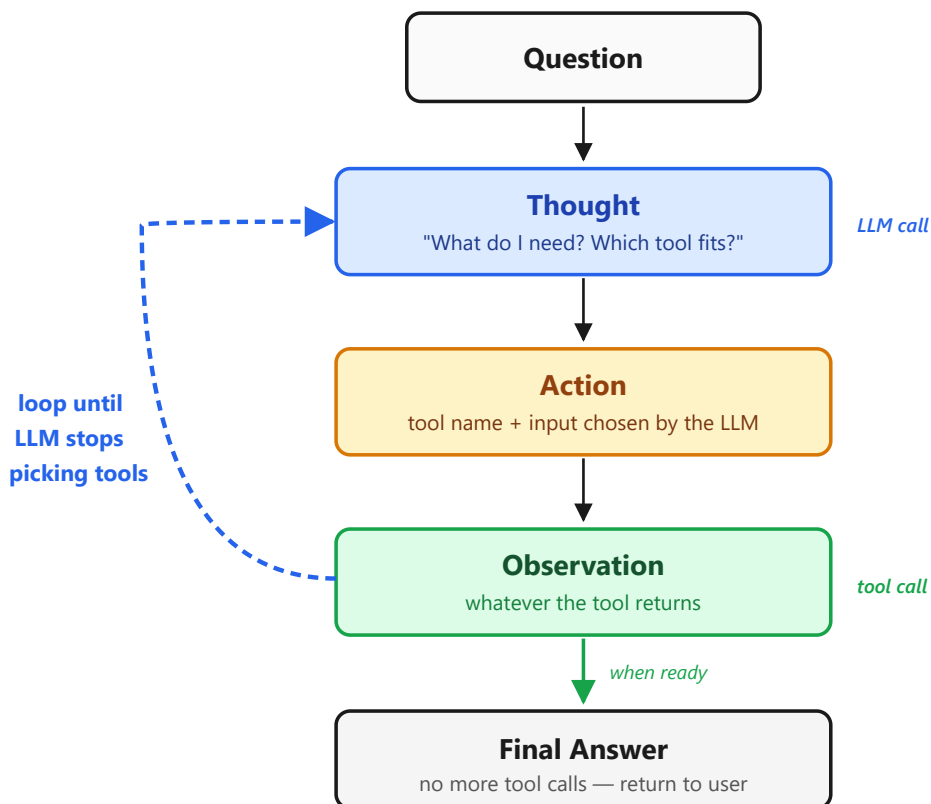
(No new tests — these are used by M7.)

M7 — `main.py`: full agent invocation

Goal: run the agent end-to-end with the demo question and view the trace in LangSmith.

The pattern the agent follows at runtime:

ReAct = Reasoning + Acting — the LLM alternates between thinking and using tools



In the `else:` branch of the `if TEST_TO_RUN is not None ...` block, write the orchestration:

1. Print a header banner with the LangSmith project name (read from `os.getenv("LANGCHAIN_PROJECT")`)
2. Call `build_pipeline()`, then `build_tools(retriever, director_sections)`, then `build_agent(tools)`
3. Save a graph picture: `agent.get_graph().draw_mermaid_png(output_file_path="agent_graph.png")` — this PNG goes into your submission
4. Print the demo question
5. Invoke the agent: `agent.invoke({"messages": [HumanMessage(content=DEMO_QUESTION)]})` — same signature as Reflection

6. The result has `result["messages"]` with the full conversation. The final answer is the last `AIMessage`. Filter the list, take the last one's `.content`, print it under a "FINAL ANSWER" banner.
7. Print a hint at the end pointing students at <https://smith.langchain.com> to find the trace URL for the project.

💡 In Reflexion you used `MAX_ITERATIONS` to bound the loop. LangGraph's prebuilt agent uses `recursion_limit` instead — you'd pass `config={"recursion_limit": 25}` to `agent.invoke(...)`. Default is 25, so you only need to set it explicitly if your agent loops longer.

Set `TEST_TO_RUN = None` and run `uv run python main.py`.

Expected (takes 2-3 minutes):

- Console scrolls through ingestion, retriever build, then the agent's reasoning
- `agent_graph.png` saved (this goes into your submission!)
- Final answer with directors + LinkedIn URLs + financial goals + auto show
- Trace appears in your LangSmith project (<https://smith.langchain.com>)

📄 **Note on "this year"**: the demo question asks about Tesla's financial goals "this year" but the 10-K data is from 2023. Your agent will likely answer with 2023 data — that's the most recent year in the filing, and it's the right behavior. You'll discuss this temporal ambiguity in your `SUBMISSION.md`.

When you're done

Open the LangSmith trace. You should see a tree of LLM calls and tool calls — Thought / Action / Observation pattern. Copy the trace URL; you'll submit it.

Copy the agent's "FINAL ANSWER" output from your console — you'll paste it into `SUBMISSION.md`.

See `../HW_INSTRUCTIONS.md` for the submission template, rubric, and common pitfalls.